Максимов Гліб К-18

Програмний проєкт 2, варіант 6: скалярний добуток (inner_product / transform_reduce)

Середовище: Intel Core i5-12400F (6 ядер, 12 потоків), Windows 11, MSVC 19.44 (Visual Studio 2022 Build Tools), /std:c++20 /EHsc /MD. Рівні оптимізації: /Od та /O2. Дані: два вектори int зі значень [-1000;1000] (mt19937, фіксоване seed), довжини 10^4, 10^5, 10^6, 10^7, збережені у текстових файлах і однакові в усіх експериментах. Час - медіана 5-200 запусків (залежно від N), мс. Результат кожного методу звіряється з послідовним inner_product.

1-2. Бібліотечні алгоритми (мс). inner_product - без політики; TR = transform_reduce з політикою

/O2

N	inner_product	TR seq	TR unseq	TR par	TR par_unseq
10^4	0.0018	0.0018	0.0018	0.0168	0.0170
10^5	0.0172	0.0298	0.0293	0.0335	0.0304
10^6	0.194	0.217	0.340	0.103	0.097
10^7	3.90	4.33	4.30	1.87	1.86

/Od

N	inner_product	TR seq	TR unseq	TR par	TR par_unseq
10^4	0.0205	0.0076	0.0079	0.0282	0.0297
10^5	0.247	0.079	0.086	0.080	0.074
10^6	2.49	0.90	0.85	0.49	0.42
10^7	24.17	11.06	11.92	3.87	3.84

3. Власний паралельний алгоритм: час (мс) залежно від K (N=10^6 і 10^7)

K	1	2	3	4	6	8	12	16	24	32	48	64	128	256	512	1024
/O2, 10^6	0.50	0.30	0.27	0.28	0.32	0.38	0.54	0.68	1.03	1.29	1.86	2.42	4.96	9.85	19.4	37.4
/O2, 10^7	4.58	2.61	2.41	2.28	2.09	1.95	2.01	1.96	2.02	2.04	2.31	2.70	5.25	10.5	22.3	42.7
/Od, 10^6	2.79	1.92	1.38	1.29	0.97	0.86	0.83	0.88	1.14	1.47	2.02	2.63	5.41	10.2	20.1	41.4
/Od, 10^7	26.8	13.4	10.2	9.35	7.56	5.71	6.71	5.85	6.13	5.70	5.87	5.98	7.44	11.3	20.8	40.0

Повні таблиці (K до 1024, усі N) - у файлах output_O2.txt, output_Od.txt.

Найкраще K	N=10^4	10^5	10^6	10^7
/O2 (K; K/12 потоків)	1 (0.08)	1 (0.08)	3 (0.25)	8 (0.67)
/Od (K; K/12 потоків)	1 (0.08)	3 (0.25)	12 (1.00)	32 (2.67), але K=8..64 практично однаково (5.7-6.0)

Висновки

- **Оптимізація:** /O2 прискорює послідовний inner_product на 10^7 у 6.2 раза (24.2 -> 3.9 мс), а паралельні політики - лише в ~2 рази (3.84 -> 1.86 мс): при /O2 цикл упирається у пропускну здатність пам'яті (80 МБ за ~4 мс, ~20 ГБ/с), тому підсилення обчислень і паралелізм дають менше. Без оптимізації TR seq вдвічі швидший за inner_product (шаблонний код transform_reduce краще розгортається навіть на /Od).
- **Політики:** на малих N (10^4, 10^5) par/par_unseq гірші за послідовний варіант (накладні витрати пулу потоків ~15-30 мкс більші за саму роботу). Вигода з'являється від 10^6 (2x на /O2, 5x на /Od при 10^6-10^7). par та par_unseq практично не відрізняються; unseq окремо не дає виграшу на /O2 (компілятор і так векторизує), на 10^6 навіть гірший.
- **Власний алгоритм, найкраще K:** залежить від N та оптимізації. Для великих N на /O2 найкраще K=8 (0.67 від 12 потоків процесора, при K=6..32 час майже плоский, 1.95-2.04 мс): далі швидкість обмежує пам'ять, а не кількість ядер. На /Od обчислення важчі, тому оптимум зсувається до K ~ кількості потоків і більше (K=12 для 10^6; плато K=8..64 для 10^7). Для малих N найкраще K=1-3: створення потоку дорожче за роботу. Власний алгоритм на /O2 (1.95 мс) на рівні par політики (1.86 мс).
- **Закон зростання часу з K:** за оптимумом час росте лінійно: T(K) ~ T0 + c*K, c ~ 0.035-0.04 мс на потік (K=256: ~10 мс, 512: ~19-22 мс, 1024: ~37-43 мс - подвоєння K подвоює час, не залежить від N). Це вартість створення та приєднання потоку (плюс зведення K часткових сум, яке мізерне). Відповідно, збільшувати K понад кількість апаратних потоків недоцільно.